

```

void removeAdjacentDuplicates(char *str){
    int stkptr=-1;
    int i=0;
    int len=strlen(str);
    while (i<len){
        if (stkptr == -1 || str[stkptr]!=str[i]){
            stkptr++;
            str[stkptr]=str[i];
            i++;
        }else {
            while(i < len&& str[stkptr]==str[i])
                i++;
            stkptr--;
        }
    }
    str[stkptr+1]='\0';
}

```

Time Complexity: $O(n)$. Space Complexity: $O(1)$ as the stack simulation is done inplace.

Problem-29 Given an array of elements, replace every element with nearest greater element on the right of that element.

Solution: One simple approach would involve scanning the array elements and for each of the elements, scan the remaining elements and find the nearest greater element.

```

void replaceWithNearestGreaterElement(int A[], int n){
    int nextNearestGreater = INT_MIN;
    int i = 0, j = 0;
    for (i=0; i<n; i++){
        nextNearestGreater = -INT_MIN;
        for (j = i+1; j<n; j++){
            if (A[i] < A[j]){
                nextNearestGreater = A[j];
                break;
            }
        }
        printf("For the element %d, %d is the nearest greater element\n", A[i], nextNearestGreater);
    }
}

```

Time Complexity: $O(n^2)$. Space Complexity: $O(1)$.

Problem-30 For [Problem-29](#), can we improve the complexity?

Solution: The approach is pretty much similar to [Problem-22](#). Create a stack and push the first element. For the rest of the elements, mark the current element as *nextNearestGreater*. If stack is not empty, then pop an element from stack and compare it with *nextNearestGreater*. If *nextNearestGreater* is greater than the popped element, then *nextNearestGreater* is the next greater element for the popped element. Keep popping from the stack while the popped element is smaller than *nextNearestGreater*. *nextNearestGreater* becomes the next greater element for all such popped elements. If *nextNearestGreater* is smaller than the popped element, then push the popped element back.

```

void replaceWithNearestGreaterElement(int A[], int n){
    int i = 0;
    struct Stack *S = CreateStack();
    int element, nextNearestGreater;
    Push(S, A[0]);
    for (i=1; i<n; i++){
        nextNearestGreater = A[i];
        if (!IsEmptyStack(S)){
            element = Pop(S);
            while (element < nextNearestGreater){
                printf("For the element %d, %d is the nearest greater element\n", A[i], nextNearestGreater);
                if(IsEmptyStack(S))
                    break;
                element = Pop(S);
            }
            if (element > nextNearestGreater)
                Push(S, element);
        }
        Push(S, nextNearestGreater);
    }
    while (!IsEmptyStack(S)){
        element = Pop(S);
        nextNearestGreater = -INT_MIN;
        printf("For the element %d, %d is the nearest greater element\n", A[i], nextNearestGreater);
    }
}

```

Time Complexity: $O(n)$. Space Complexity: $O(n)$.

Problem-31 How to implement a stack which will support following operations in $O(1)$ time complexity?

- Push which adds an element to the top of stack.
- Pop which removes an element from top of stack.
- Find Middle which will return middle element of the stack.
- Delete Middle which will delete the middle element.

Solution: We can use a LinkedList data structure with an extra pointer to the middle element.

Also, we need another variable to store whether the LinkedList has an even or odd number of elements.

- *Push*: Add the element to the head of the LinkedList. Update the pointer to the middle element according to variable.
- *Pop*: Remove the head of the LinkedList. Update the pointer to the middle element according to variable.
- *Find Middle*: Find Middle which will return middle element of the stack.
- *Delete Middle*: Delete Middle which will delete the middle element use the logic of [Problem-43](#) from *Linked Lists* chapter.

QUEUES

CHAPTER

5



5.1 What is a Queue?

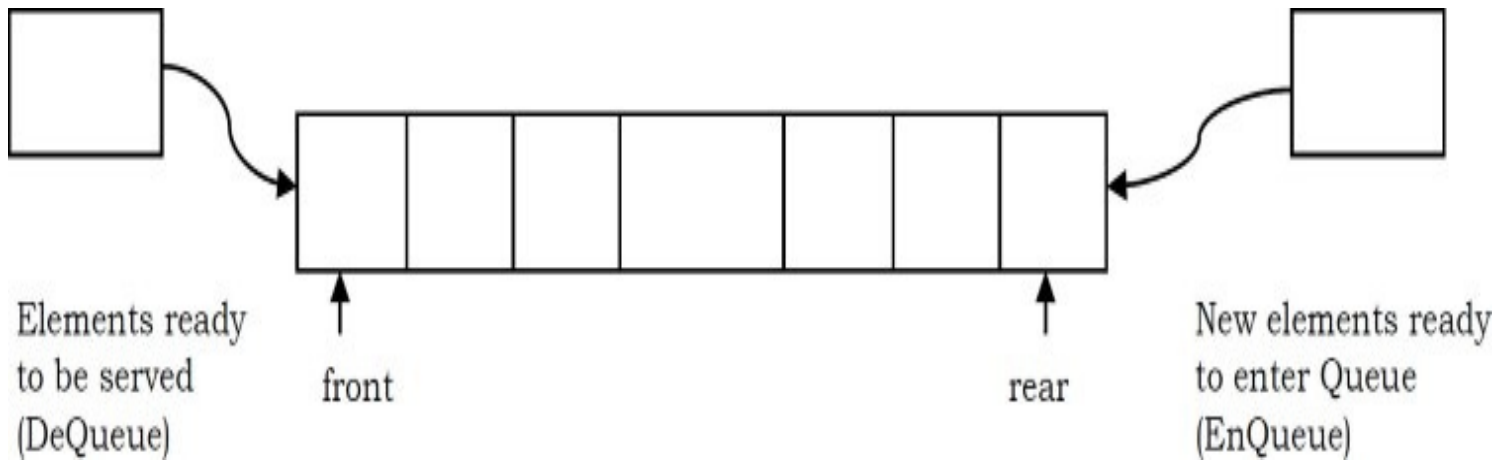
A queue is a data structure used for storing data (similar to Linked Lists and Stacks). In queue, the order in which data arrives is important. In general, a queue is a line of people or things waiting to be served in sequential order starting at the beginning of the line or sequence.

Definition: A *queue* is an ordered list in which insertions are done at one end (*rear*) and deletions are done at other end (*front*). The first element to be inserted is the first one to be deleted. Hence, it is called First in First out (FIFO) or Last in Last out (LILO) list.

Similar to *Stacks*, special names are given to the two changes that can be made to a queue. When an element is inserted in a queue, the concept is called *EnQueue*, and when an element is removed from the queue, the concept is called *DeQueue*.

DeQueueing an empty queue is called *underflow* and *EnQueueing* an element in a full queue is called *overflow*. Generally, we treat them as exceptions. As an example, consider the snapshot of

the queue.



5.2 How are Queues Used?

The concept of a queue can be explained by observing a line at a reservation counter. When we enter the line we stand at the end of the line and the person who is at the front of the line is the one who will be served next. He will exit the queue and be served.

As this happens, the next person will come at the head of the line, will exit the queue and will be served. As each person at the head of the line keeps exiting the queue, we move towards the head of the line. Finally we will reach the head of the line and we will exit the queue and be served. This behavior is very useful in cases where there is a need to maintain the order of arrival.

5.3 Queue ADT

The following operations make a queue an ADT. Insertions and deletions in the queue must follow the FIFO scheme. For simplicity we assume the elements are integers.

Main Queue Operations

- `EnQueue(int data)`: Inserts an element at the end of the queue
- `int DeQueue()`: Removes and returns the element at the front of the queue

Auxiliary Queue Operations

- `int Front()`: Returns the element at the front without removing it
- `int QueueSize()`: Returns the number of elements stored in the queue
- `int IsEmptyQueueQ`: Indicates whether no elements are stored in the queue or not

5.4 Exceptions

Similar to other ADTs, executing *DeQueue* on an empty queue throws an “*Empty Queue Exception*” and executing *EnQueue* on a full queue throws “*Full Queue Exception*”.

5.5 Applications

Following are some of the applications that use queues.

Direct Applications

- Operating systems schedule jobs (with equal priority) in the order of arrival (e.g., a print queue).
- Simulation of real-world queues such as lines at a ticket counter or any other first-come first-served scenario requires a queue.
- Multiprogramming.
- Asynchronous data transfer (file IO, pipes, sockets).
- Waiting times of customers at call center.
- Determining number of cashiers to have at a supermarket.

Indirect Applications

- Auxiliary data structure for algorithms
- Component of other data structures

5.6 Implementation

There are many ways (similar to Stacks) of implementing queue operations and some of the commonly used methods are listed below.

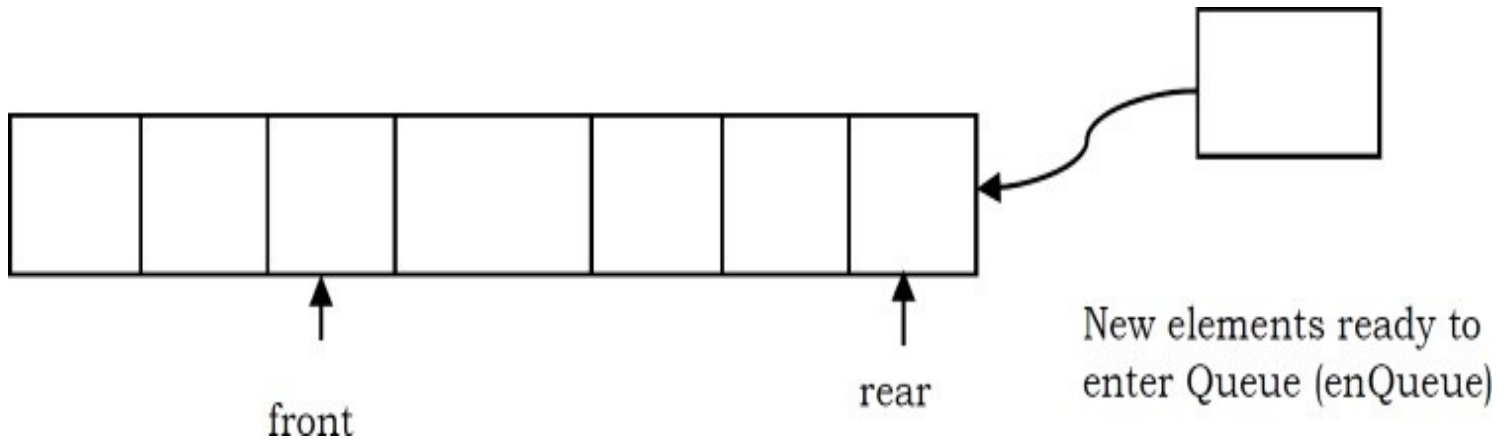
- Simple circular array based implementation
- Dynamic circular array based implementation
- Linked list implementation

Why Circular Arrays?

First, let us see whether we can use simple arrays for implementing queues as we have done for stacks. We know that, in queues, the insertions are performed at one end and deletions are performed at the other end. After performing some insertions and deletions the process becomes easy to understand.

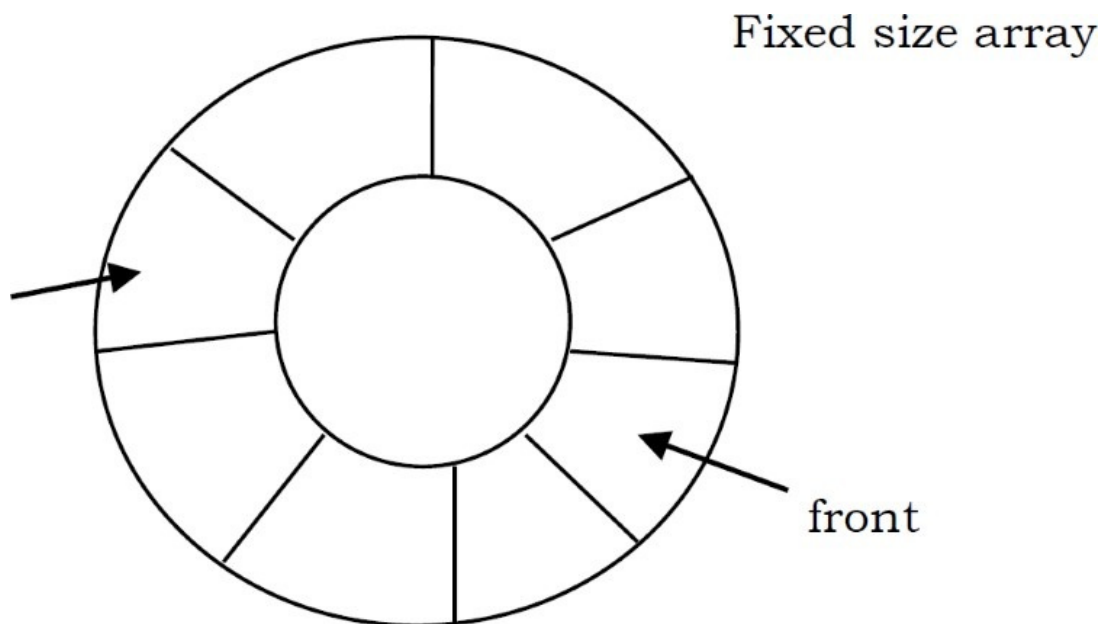
In the example shown below, it can be seen clearly that the initial slots of the array are getting wasted. So, simple array implementation for queue is not efficient. To solve this problem we assume the arrays as circular arrays. That means, we treat the last element and the first array

elements as contiguous. With this representation, if there are any free slots at the beginning, the rear pointer can easily go to its next free slot.



Note: The simple circular array and dynamic circular array implementations are very similar to stack array implementations. Refer to [Stacks](#) chapter for analysis of these implementations.

Simple Circular Array Implementation



This simple implementation of Queue ADT uses an array. In the array, we add elements circularly and use two variables to keep track of the start element and end element. Generally, *front* is used to indicate the start element and *rear* is used to indicate the end element in the queue. The array storing the queue elements may become full. An *EnQueue* operation will then throw a *full queue exception*. Similarly, if we try deleting an element from an empty queue it will throw *empty queue exception*.

Note: Initially, both front and rear points to -1 which indicates that the queue is empty.


```

struct ArrayQueue {
    int front, rear;
    int capacity;
    int *array;
};

struct ArrayQueue *Queue(int size) {
    struct ArrayQueue *Q = malloc(sizeof(struct ArrayQueue));
    if(!Q)
        return NULL;
    Q->capacity = size;
    Q->front = Q->rear = -1;
    Q->array = malloc(Q->capacity * sizeof(int));
    if(!Q->array)
        return NULL;
    return Q;
}

int IsEmptyQueue(struct ArrayQueue *Q) {
    // if the condition is true then 1 is returned else 0 is returned
    return (Q->front == -1);
}

int IsFullQueue(struct ArrayQueue *Q) {
    //if the condition is true then 1 is returned else 0 is returned
    return ((Q->rear + 1) % Q->capacity == Q->front);
}

int QueueSize() {
    return (Q->capacity - Q->front + Q->rear + 1)% Q->capacity;
}

void EnQueue(struct ArrayQueue *Q, int data) {
    if(IsFullQueue(Q))
        printf("Queue Overflow");
    else {
        Q->rear = (Q->rear+1) % Q->capacity;
        Q->array[Q->rear] = data;
        if(Q->front == -1)
            Q->front = Q->rear;
    }
}

int DeQueue(struct ArrayQueue *Q) {
    int data = 0; //or element which does not exist in Queue
    if(IsEmptyQueue(Q)) {
        printf("Queue is Empty");
        return 0;
    }
    else {
        data = Q->array[Q->front];
        if(Q->front == Q->rear)
            Q->front = Q->rear = -1;
        else Q->front = (Q->front+1) % Q->capacity;
    }
    return data;
}

void DeleteQueue(struct ArrayQueue *Q) {
    if(Q) {
        if(Q->array)
            free(Q->array);
        free(Q);
    }
}

```

Performance and Limitations

Performance: Let n be the number of elements in the queue:

Space Complexity (for n EnQueue operations)	$O(n)$
Time Complexity of EnQueue()	$O(1)$
Time Complexity of DeQueue()	$O(1)$
Time Complexity of IsEmptyQueue()	$O(1)$
Time Complexity of IsFullQueue()	$O(1)$
Time Complexity of QueueSize()	$O(1)$
Time Complexity of DeleteQueue()	$O(1)$

Limitations: The maximum size of the queue must be defined as prior and cannot be changed. Trying to *EnQueue* a new element into a full queue causes an implementation-specific exception.

Dynamic Circular Array Implementation

```

struct DynArrayQueue {
    int front, rear;
    int capacity;
    int *array;
};

struct DynArrayQueue *CreateDynQueue() {
    struct DynArrayQueue *Q = malloc(sizeof(struct DynArrayQueue));
    if(!Q)
        return NULL;
    Q->capacity = 1;
    Q->front = Q->rear = -1;
    Q->array = malloc(Q->capacity * sizeof(int));
    if(!Q->array)
        return NULL;
    return Q;
}

int IsEmptyQueue(struct DynArrayQueue *Q) {
    // if the condition is true then 1 is returned else 0 is returned
    return (Q->front == -1);
}

int IsFullQueue(struct DynArrayQueue *Q) {
    //if the condition is true then 1 is returned else 0 is returned
    return ((Q->rear + 1) % Q->capacity == Q->front);
}

int QueueSize() {
    return (Q->capacity - Q->front + Q->rear + 1)% Q->capacity;
}

void EnQueue(struct DynArrayQueue *Q, int data) {
    if(IsFullQueue(Q))
        ResizeQueue(Q);
    Q->rear = (Q->rear+1)% Q->capacity;
    Q->array[Q->rear] = data;

    if(Q->front == -1)
        Q->front = Q->rear;
}

void ResizeQueue(struct DynArrayQueue *Q) {
    int size = Q->capacity;
    Q->capacity = Q->capacity*2;
    Q->array = realloc (Q->array, Q->capacity);
    if(!Q->array) {
        printf("Memory Error");
        return;
    }
    if(Q->front > Q->rear ) {
        for(int i=0; i < Q->front; i++) {
            Q->array[i+size] = Q->array[i];
        }
        Q->rear = Q->rear + size;
    }
}

int DeQueue(struct DynArrayQueue *Q) {
    int data = 0; // or element which does not exist in Queue
    if(IsEmptyQueue(Q)) {
        printf("Queue is Empty");
        return 0;
    }
    else {
        data = Q->array[Q->front];
        if(Q->front== Q->rear)
            Q->front= Q->rear = -1;
        else
            Q->front = (Q->front+1) % Q->capacity;
    }
    return data;
}

void DeleteQueue(struct DynArrayQueue *Q) {
    if(Q) {
        if(Q->array)
            free(Q->array);
        free(Q->array);
    }
}

```

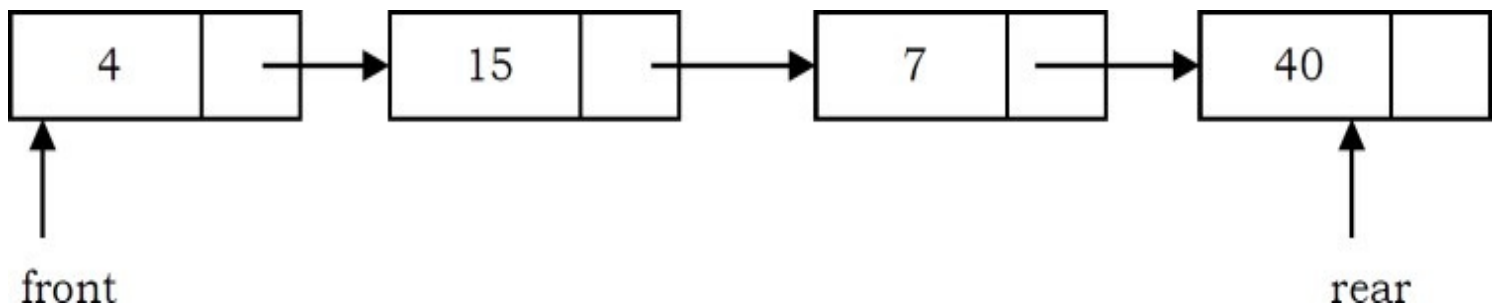
Performance

Let n be the number of elements in the queue.

Space Complexity (for n EnQueue operations)	$O(n)$
Time Complexity of EnQueue()	$O(1)$ (Average)
Time Complexity of DeQueue()	$O(1)$
Time Complexity of QueueSize()	$O(1)$
Time Complexity of IsEmptyQueue()	$O(1)$
Time Complexity of IsFullQueue()	$O(1)$
Time Complexity of QueueSize()	$O(1)$
Time Complexity of DeleteQueue()	$O(1)$

Linked List Implementation

Another way of implementing queues is by using Linked lists. *EnQueue* operation is implemented by inserting an element at the end of the list. *DeQueue* operation is implemented by deleting an element from the beginning of the list.



```
struct ListNode {  
    int data;  
    struct ListNode *next;  
};
```

```
struct Queue {  
    struct ListNode *front;  
    struct ListNode *rear;  
};
```

```

struct Queue *CreateQueue() {
    struct Queue *Q;
    struct ListNode *temp;
    Q = malloc(sizeof(struct Queue));
    if(!Q)
        return NULL;
    temp = malloc(sizeof(struct ListNode));
    Q→front = Q→rear = NULL;
    return Q;
}

int IsEmptyQueue(struct Queue *Q) {
    // if the condition is true then 1 is returned else 0 is returned
    return (Q→front == NULL);
}

void EnQueue(struct Queue *Q, int data) {
    struct ListNode *newNode;
    newNode = malloc(sizeof(struct ListNode));
    if(!newNode)
        return NULL;
    newNode→data = data;
    newNode→next = NULL;
    if(Q→rear) Q→rear→next = newNode;
    Q→rear = newNode;

    if(Q→front == NULL)
        Q→front = Q→rear;
}

int DeQueue(struct Queue *Q) {
    int data = 0;    //or element which does not exist in Queue
    struct ListNode *temp;
    if(IsEmptyQueue(Q)) {
        printf("Queue is empty");
        return 0;
    }
    else {
        temp = Q→front;
        data = Q→front→data;
        Q→front = Q→front→next;
        free(temp);
    }
    return data;
}

void DeleteQueue(struct Queue *Q) {
    struct ListNode *temp;
    while(Q) {
        temp = Q;
        Q = Q→next;
        free(temp);
    }
    free(Q);
}

```

Performance

Let n be the number of elements in the queue, then

Space Complexity (for n EnQueue operations)	$O(n)$
Time Complexity of EnQueue()	$O(1)$ (Average)
Time Complexity of DeQueue()	$O(1)$
Time Complexity of IsEmptyQueue()	$O(1)$
Time Complexity of DeleteQueue()	$O(1)$

Comparison of Implementations

Note: Comparison is very similar to stack implementations and *Stacks* chapter.

5.7 Queues: Problems & Solutions

Problem-1 Give an algorithm for reversing a queue Q . To access the queue, we are only allowed to use the methods of queue ADT.

Solution:

```
void ReverseQueue(struct Queue *Q) {  
    struct Stack *S = CreateStack();  
    while (!IsEmptyQueue(Q))  
        Push(S, DeQueue(Q))  
    while (!IsEmptyStack(S))  
        EnQueue(Q, Pop(S));  
}
```

Time Complexity: $O(n)$.

Problem-2 How can you implement a queue using two stacks?

Solution: Let $S1$ and $S2$ be the two stacks to be used in the implementation of queue. All we have to do is to define the EnQueue and DeQueue operations for the queue.

```

struct Queue {
    struct Stack *S1; // for EnQueue
    struct Stack *S2; // for DeQueue
}

```

EnQueue Algorithm

- Just push on to stack S1

```

void EnQueue(struct Queue *Q, int data) {
    Push(Q→S1, data);
}

```

Time Complexity: $O(1)$.

DeQueue Algorithm

- If stack S2 is not empty then pop from S2 and return that element.
- If stack is empty, then transfer all elements from S1 to S2 and pop the top element from S2 and return that popped element [we can optimize the code a little by transferring only $n - 1$ elements from S1 to S2 and pop the n^{th} element from S1 and return that popped element].
- If stack S1 is also empty then throw error.

```

int DeQueue(struct Queue *Q) {
    if(!IsEmptyStack(Q→S2))
        return Pop(Q→S2);
    else {
        while(!IsEmptyStack(Q→S1))
            Push(Q→S2, Pop(Q→S1));
        return Pop(Q→S2);
    }
}

```

Time Complexity: From the algorithm, if the stack S2 is not empty then the complexity is $O(1)$. If the stack S2 is empty, then we need to transfer the elements from S1 to S2. But if we carefully observe, the number of transferred elements and the number of popped elements from S2 are equal. Due to this the average complexity of pop operation in this case is $O(1)$. The amortized complexity of pop operation is $O(1)$.

Problem-3 Show how you can efficiently implement one stack using two queues. Analyze the

running time of the stack operations.

Solution: Yes, it is possible to implement the Stack ADT using 2 implementations of the Queue ADT. One of the queues will be used to store the elements and the other to hold them temporarily during the *pop* and *top* methods. The *push* method would *enqueue* the given element onto the storage queue. The *top* method would transfer all but the last element from the storage queue onto the temporary queue, save the front element of the storage queue to be returned, transfer the last element to the temporary queue, then transfer all elements back to the storage queue. The *pop* method would do the same as *top*, except instead of transferring the last element onto the temporary queue after saving it for return, that last element would be discarded. Let Q1 and Q2 be the two queues to be used in the implementation of stack. All we have to do is to define the *push* and *pop* operations for the stack.

```
struct Stack {  
    struct Queue *Q1;  
    struct Queue *Q2;  
}
```

In the algorithms below, we make sure that one queue is always empty.

Push Operation Algorithm: Insert the element in whichever queue is not empty.

- Check whether queue Q1 is empty or not. If Q1 is empty then Enqueue the element into Q2.
- Otherwise EnQueue the element into Q1.

```
Push(struct Stack *S, int data) {  
    if(IsEmptyQueue(S→Q1))  
        EnQueue(S→Q2, data);  
    else    EnQueue(S→Q1, data);  
}
```

Time Complexity: $O(1)$.

Pop Operation Algorithm: Transfer $n - 1$ elements to the other queue and delete last from queue for performing pop operation.

- If queue Q1 is not empty then transfer $n - 1$ elements from Q1 to Q2 and then, DeQueue the last element of Q1 and return it.
- If queue Q2 is not empty then transfer $n - 1$ elements from Q2 to Q1 and then, DeQueue the last element of Q2 and return it.


```

int Pop(struct Stack *S) {
    int i, size;
    if(IsEmptyQueue(S→Q2)) {
        size = Size(S→Q1);
        i = 0;
        while(i < size-1) {
            EnQueue(S→Q2, DeQueue(S→Q1));
            i++;
        }
        return DeQueue(S→Q1);
    }
    else {
        size = Size(S→Q2);
        while(i < size-1) {
            EnQueue(S→Q1, DeQueue(S→Q2));
            i++;
        }
        return DeQueue(S→Q2);
    }
}

```

Time Complexity: Running time of pop operation is $O(n)$ as each time pop is called, we are transferring all the elements from one queue to the other.

Problem-4 Maximum sum in sliding window: Given array $A[]$ with sliding window of size w which is moving from the very left of the array to the very right. Assume that we can only see the w numbers in the window. Each time the sliding window moves rightwards by one position. For example: The array is $[1\ 3\ -1\ -3\ 5\ 3\ 6\ 7]$, and w is 3.

Window position	Max
$[1\ 3\ -1]\ -3\ 5\ 3\ 6\ 7$	3
$1\ [3\ -1\ -3]\ 5\ 3\ 6\ 7$	3
$1\ 3\ [-1\ -3\ 5]\ 3\ 6\ 7$	5
$1\ 3\ -1\ [-3\ 5\ 3]\ 6\ 7$	5
$1\ 3\ -1\ -3\ [5\ 3\ 6]\ 7$	6
$1\ 3\ -1\ -3\ 5\ [3\ 6\ 7]$	7

Input: A long array $A[]$, and a window width w . **Output:** An array $B[]$, $B[i]$ is the maximum value from $A[i]$ to $A[i+w-1]$. **Requirement:** Find a good optimal way to get $B[i]$

Solution: This problem can be solved with doubly ended queue (which supports insertion and deletion at both ends). Refer [Priority Queues](#) chapter for algorithms.

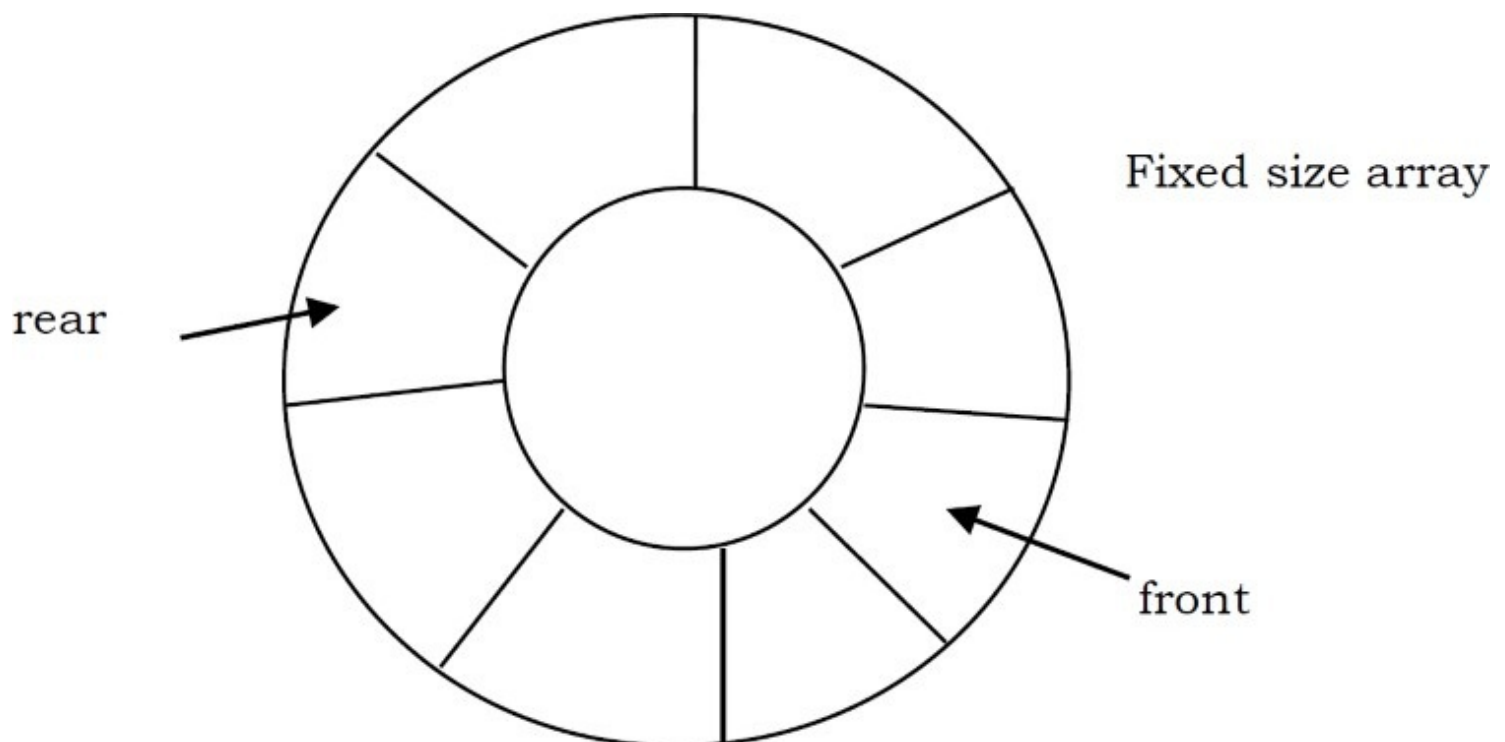
Problem-5 Given a queue Q containing n elements, transfer these items on to a stack S (initially empty) so that front element of Q appears at the top of the stack and the order of all other items is preserved. Using enqueue and dequeue operations for the queue, and push and pop operations for the stack, outline an efficient $O(n)$ algorithm to accomplish the above task, using only a constant amount of additional storage.

Solution: Assume the elements of queue Q are $a_1, a_2 \dots a_n$. Dequeueing all elements and pushing them onto the stack will result in a stack with a_n at the top and a_1 at the bottom. This is done in $O(n)$ time as dequeue and each push require constant time per operation. The queue is now empty. By popping all elements and pushing them on the queue we will get a_1 at the top of the stack. This is done again in $O(n)$ time.

As in big-oh arithmetic we can ignore constant factors. The process is carried out in $O(n)$ time. The amount of additional storage needed here has to be big enough to temporarily hold one item.

Problem-6 A queue is set up in a circular array $A[0..n-1]$ with front and rear defined as usual. Assume that $n-1$ locations in the array are available for storing the elements (with the other element being used to detect full/empty condition). Give a formula for the number of elements in the queue in terms of $rear$, $front$, and n .

Solution: Consider the following figure to get a clear idea of the queue.



- Rear of the queue is somewhere clockwise from the front.
- To enqueue an element, we move *rear* one position clockwise and write the element in that position.
- To dequeue, we simply move *front* one position clockwise.
- Queue migrates in a clockwise direction as we enqueue and dequeue.
- Emptiness and fullness to be checked carefully.
- Analyze the possible situations (make some drawings to see where *front* and *rear* are when the queue is empty, and partially and totally filled). We will get this:

$$\text{Number Of Elements} = \begin{cases} \text{rear} - \text{front} + 1 & \text{if rear} == \text{front} \\ \text{rear} - \text{front} + n & \text{otherwise} \end{cases}$$

Problem-7 What is the most appropriate data structure to print elements of queue in reverse order?

Solution: Stack.

Problem-8 Implement doubly ended queues. A double-ended queue is an abstract data structure that implements a queue for which elements can only be added to or removed from the front (head) or back (tail). It is also often called a head-tail linked list.

Solution:

```

void pushBackDEQ(struct ListNode **head, int data){
    struct ListNode *newNode = (struct ListNode*) malloc(sizeof(struct ListNode));
    newNode->data = data;
    if(*head == NULL){
        *head = newNode;
        (*head)->next = *head;
        (*head)->prev = *head;
    }
    else{
        newNode->prev = (*head)->prev;
        newNode->next = *head;
        (*head)->prev->next = newNode;
        (*head)->prev = newNode;
    }
}

void pushFrontDEQ(struct ListNode **head, int data){
    pushBackDEQ(head,data);
    *head = (*head)->prev;
}

int popBackDEQ(struct ListNode **head){
    int data;
    if( (*head)->prev == *head ){
        data = (*head)->data;
        free(*head);
        *head = NULL;
    }
    else{
        struct ListNode *newTail = (*head)->prev->prev;
        data = (*head)->prev->data;
        newTail->next = *head;
        free((*head)->prev);
        (*head) ->prev = newTail;
    }
    return data;
}

int popFront(struct ListNode **head){
    int data;
    *head = (*head)->next;
    data = popBackDEQ(head);
    return data;
}

```

Problem-9 Given a stack of integers, how do you check whether each successive pair of numbers in the stack is consecutive or not. The pairs can be increasing or decreasing, and if the stack has an odd number of elements, the element at the top is left out of a pair. For example, if the stack of elements are [4, 5, -2, -3, 11, 10, 5, 6, 20], then the output should be true because each of the pairs (4, 5), (-2, -3), (11, 10), and (5, 6) consists of consecutive numbers.

Solution:

```
int checkStackPairwiseOrder(struct Stack *s) {
    struct Queue *q = CreateQueue();
    int pairwiseOrdered = 1;
    while (!isEmptyStack(s))
        EnQueue(q, Pop(s));
    while (!IsEmptyQueue(q))
        Push(s, DeQueue(q));
    while (!isEmptyStack(s)) {
        int n = Pop(s);
        EnQueue(q, n);
        if (isEmptyStack(s)) {
            int m = Pop(s);
            EnQueue(q, m);
            if (abs(n - m) != 1) {
                pairwiseOrdered = 0;
            }
        }
    }
    while (!IsEmptyQueue(q))
        Push(s, DeQueue(q));
    return pairwiseOrdered;
}
```

Time Complexity: $O(n)$. Space Complexity: $O(n)$.

Problem-10 Given a queue of integers, rearrange the elements by interleaving the first half of the list with the second half of the list. For example, suppose a queue stores the following sequence of values: [11, 12, 13, 14, 15, 16, 17, 18, 19, 20]. Consider the two halves of this list: first half: [11, 12, 13, 14, 15] second half: [16, 17, 18, 19, 20]. These are

combined in an alternating fashion to form a sequence of interleave pairs: the first values from each half (11 and 16), then the second values from each half (12 and 17), then the third values from each half (13 and 18), and so on. In each pair, the value from the first half appears before the value from the second half. Thus, after the call, the queue stores the following values: [11, 16, 12, 17, 13, 18, 14, 19, 15, 20].

Solution:

```
void interLeavingQueue(struct Queue *q) {
    if (Size(q) % 2 != 0)
        return;
    struct Stack *s = CreateStack();
    int halfSize = Size(q) / 2;
    for (int i = 0; i < halfSize; i++)
        Push(s, DeQueue(q));
    while (!isEmptyStack(s))
        EnQueue(q, Pop(s));
    for (int i = 0; i < halfSize; i++)
        EnQueue(q, DeQueue(q));
    for (int i = 0; i < halfSize; i++)
        Push(s, DeQueue(q));
    while (!isEmptyStack(s)) {
        EnQueue(q, Pop(s));
        EnQueue(q, DeQueue(q));
    }
}
```

Time Complexity: $O(n)$. Space Complexity: $O(n)$.

Problem-11 Given an integer k and a queue of integers, how do you reverse the order of the first k elements of the queue, leaving the other elements in the same relative order? For example, if $k=4$ and queue has the elements [10, 20, 30, 40, 50, 60, 70, 80, 90]; the output should be [40, 30, 20, 10, 50, 60, 70, 80, 90].

Solution:

```

void reverseQueueFirstKElements(int k, struct Queue *q) {
    if (q == Null || k > Size(q)) {
        return;
    }
    else if (k > 0) {
        struct Stack *s = CreateStack();
        for (int i = 0; i < k; i++) {
            Push(s, DeQueue(q));
        }
        while (!isEmptyStack(s)) {
            EnQueue(q, Pop(s));
        }
        for (int i = 0; i < Size(q) - k; i++) { // wrap around rest of elements
            EnQueue(q, DeQueue(q));
        }
    }
}

```

Time Complexity: $O(n)$. Space Complexity: $O(n)$.

TREES

CHAPTER

6

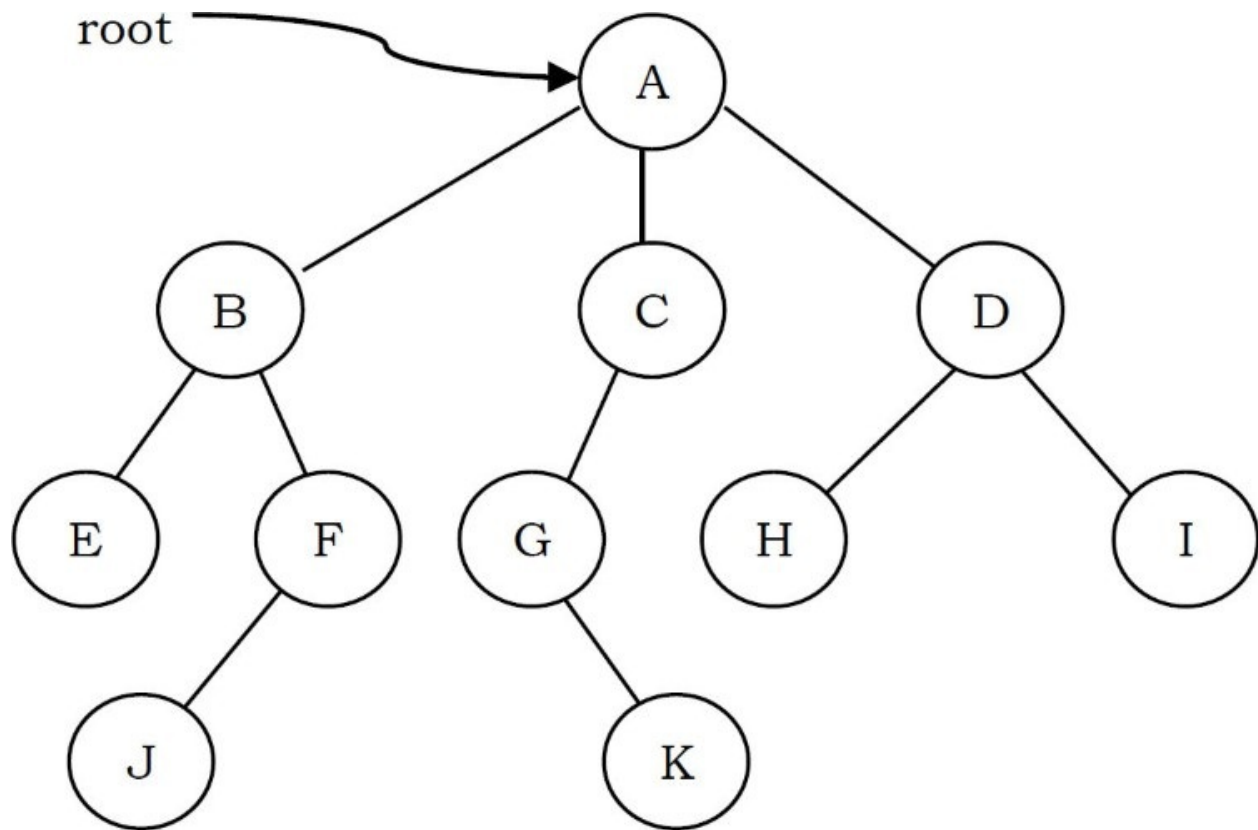


6.1 What is a Tree?

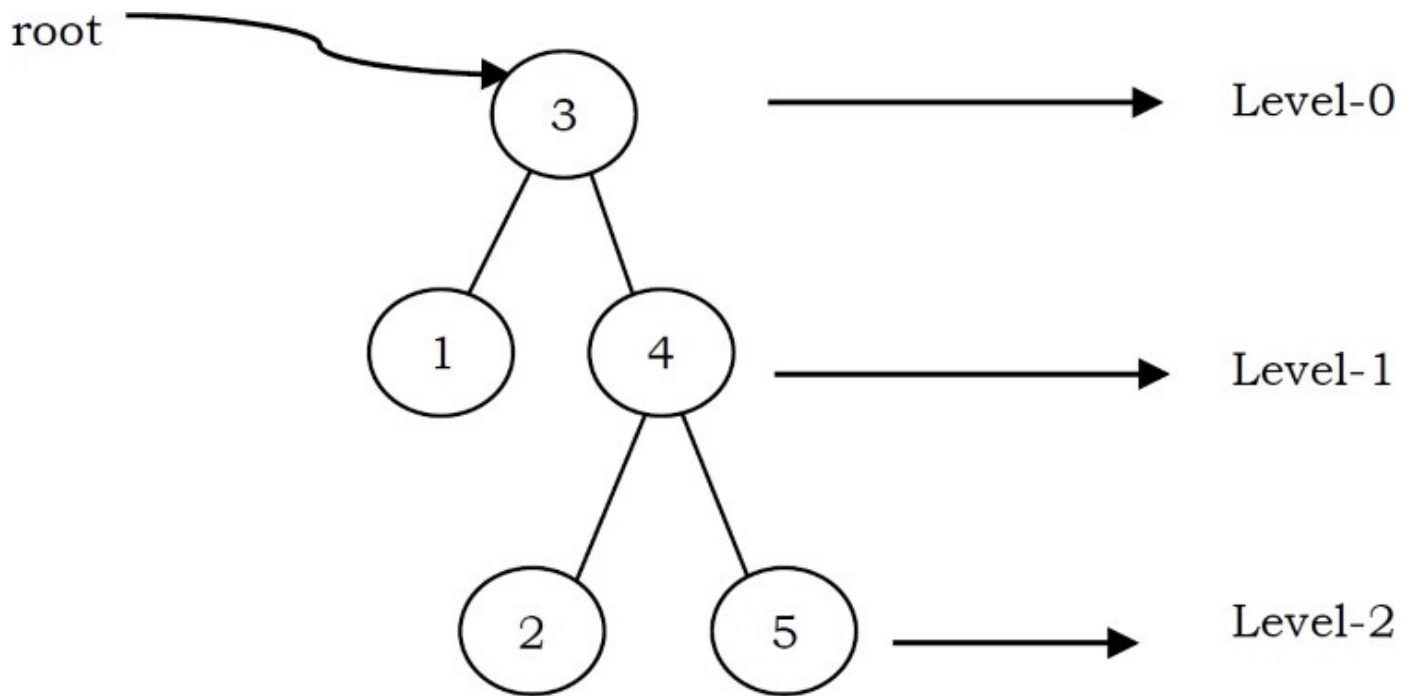
A *tree* is a data structure similar to a linked list but instead of each node pointing simply to the next node in a linear fashion, each node points to a number of nodes. Tree is an example of a non-linear data structure. A *tree* structure is a way of representing the hierarchical nature of a structure in a graphical form.

In trees ADT (Abstract Data Type), the order of the elements is not important. If we need ordering information, linear data structures like linked lists, stacks, queues, etc. can be used.

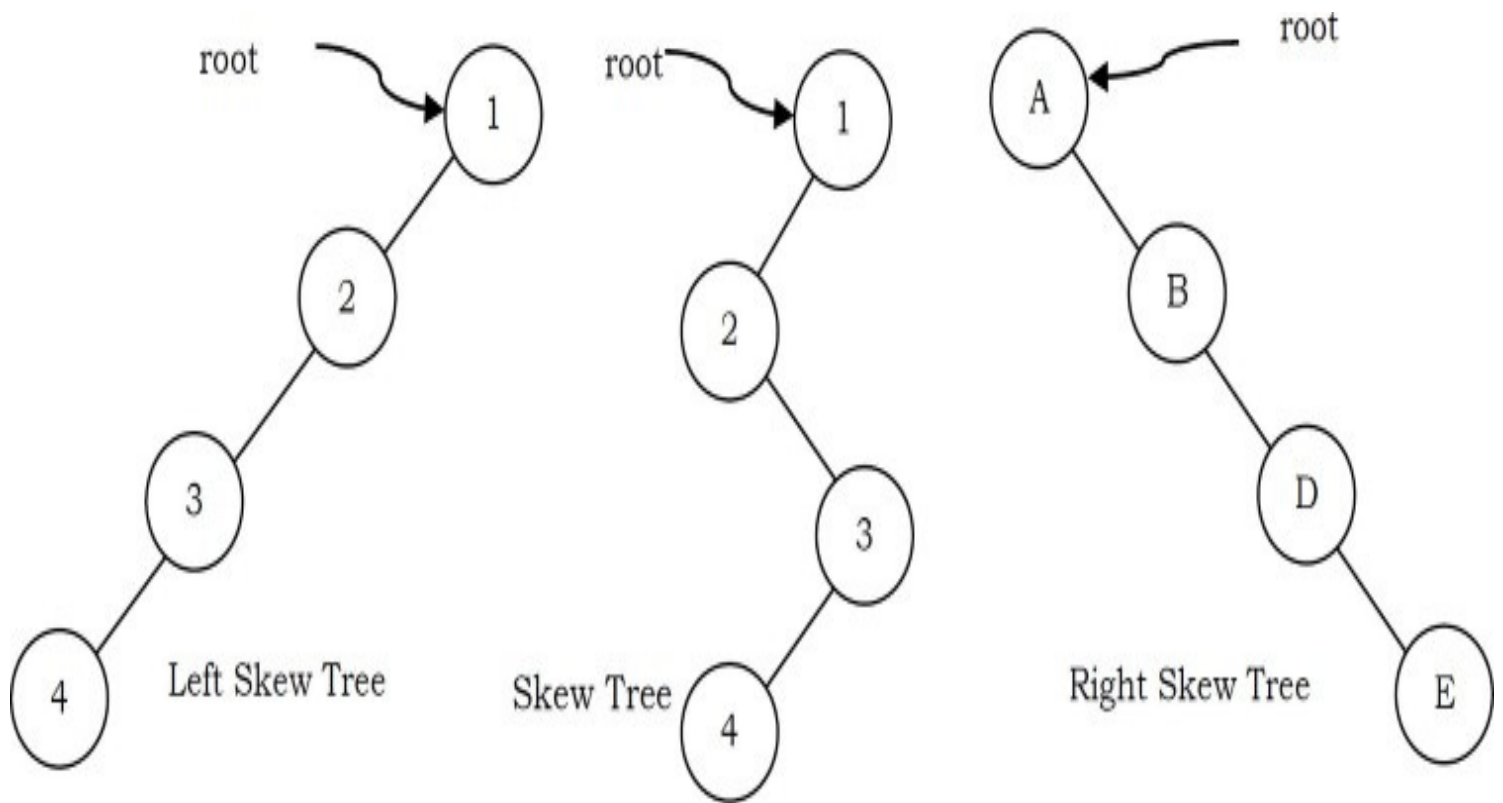
6.2 Glossary



- The *root* of a tree is the node with no parents. There can be at most one root node in a tree (node *A* in the above example).
- An *edge* refers to the link from parent to child (all links in the figure).
- A node with no children is called *leaf* node (*E, J, K, H* and *I*).
- Children of same parent are called *siblings* (*B, C, D* are siblings of *A*, and *E, F* are the siblings of *B*).
- A node *p* is an *ancestor* of node *q* if there exists a path from *root* to *q* and *p* appears on the path. The node *q* is called a *descendant* of *p*. For example, *A, C* and *G* are the ancestors of *I*.
- The set of all nodes at a given depth is called the *level* of the tree (*B, C* and *D* are the same level). The root node is at level zero.



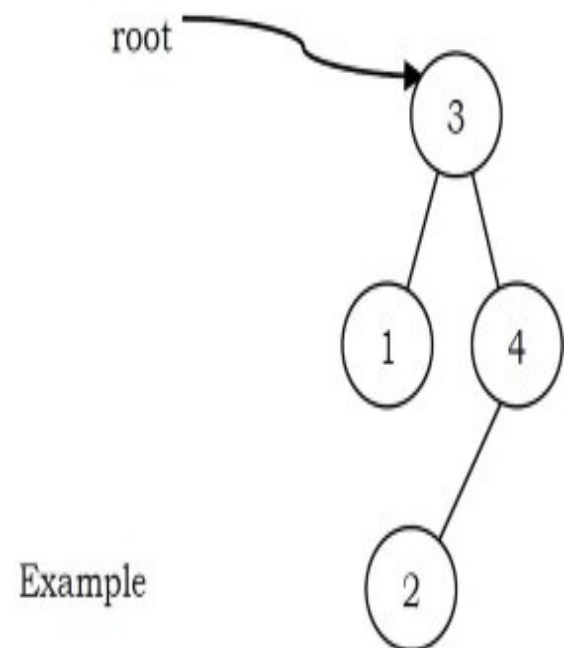
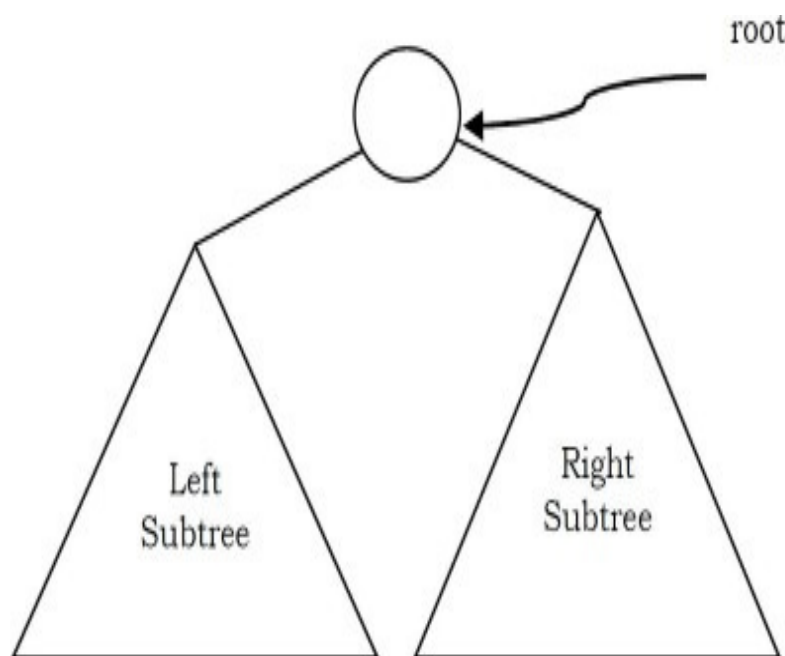
- The *depth* of a node is the length of the path from the root to the node (depth of G is 2, $A - C - G$).
- The *height* of a node is the length of the path from that node to the deepest node. The height of a tree is the length of the path from the root to the deepest node in the tree. A (rooted) tree with only one node (the root) has a height of zero. In the previous example, the height of B is 2 ($B - F - J$).
- *Height of the tree* is the maximum height among all the nodes in the tree and *depth of the tree* is the maximum depth among all the nodes in the tree. For a given tree, depth and height returns the same value. But for individual nodes we may get different results.
- The size of a node is the number of descendants it has including itself (the size of the subtree C is 3).
- If every node in a tree has only one child (except leaf nodes) then we call such trees *skew trees*. If every node has only left child then we call them *left skew trees*. Similarly, if every node has only right child then we call them *right skew trees*.



6.3 Binary Trees

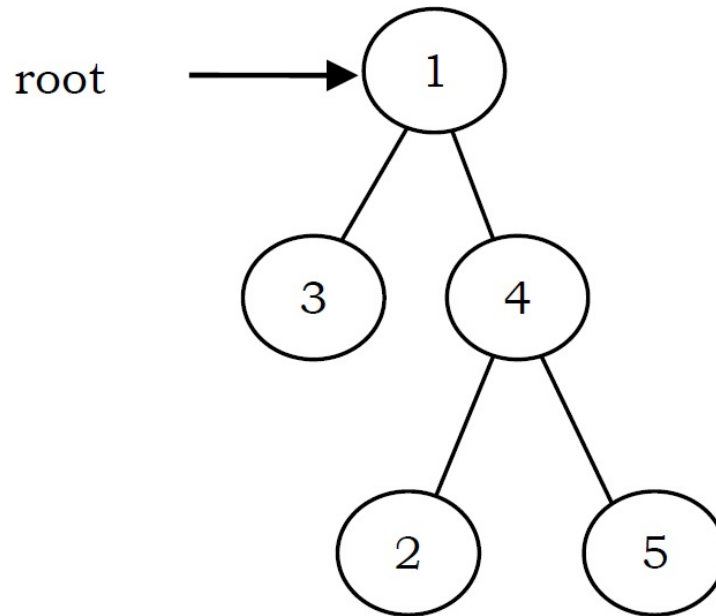
A tree is called *binary tree* if each node has zero child, one child or two children. Empty tree is also a valid binary tree. We can visualize a binary tree as consisting of a root and two disjoint binary trees, called the left and right subtrees of the root.

Generic Binary Tree

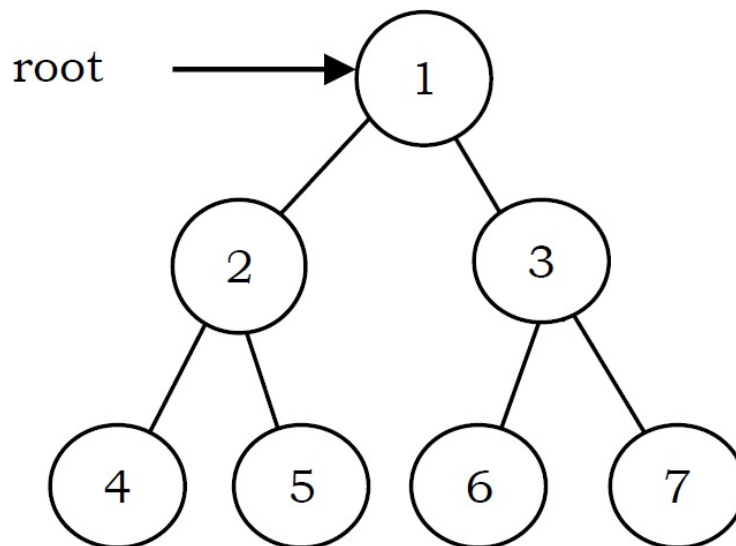


6.4 Types of Binary Trees

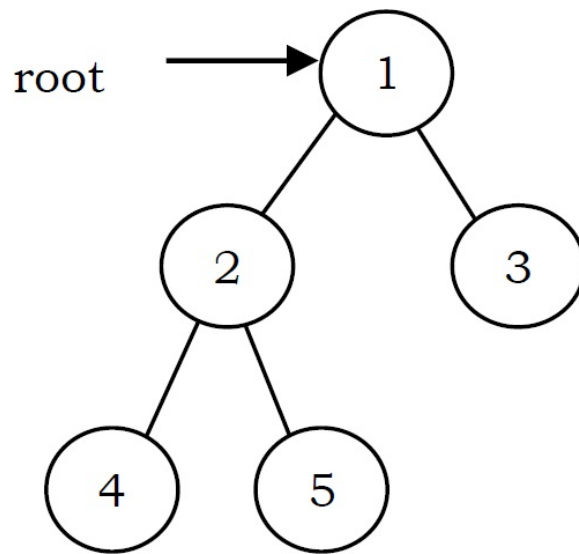
Strict Binary Tree: A binary tree is called *strict binary tree* if each node has exactly two children or no children.



Full Binary Tree: A binary tree is called *full binary tree* if each node has exactly two children and all leaf nodes are at the same level.



Complete Binary Tree: Before defining the *complete binary tree*, let us assume that the height of the binary tree is h . In complete binary trees, if we give numbering for the nodes by starting at the root (let us say the root node has 1) then we get a complete sequence from 1 to the number of nodes in the tree. While traversing we should give numbering for NULL pointers also. A binary tree is called *complete binary tree* if all leaf nodes are at height h or $h - 1$ and also without any missing number in the sequence.



6.5 Properties of Binary Trees

For the following properties, let us assume that the height of the tree is h . Also, assume that root node is at height zero.

Height**Number of nodes at level h**

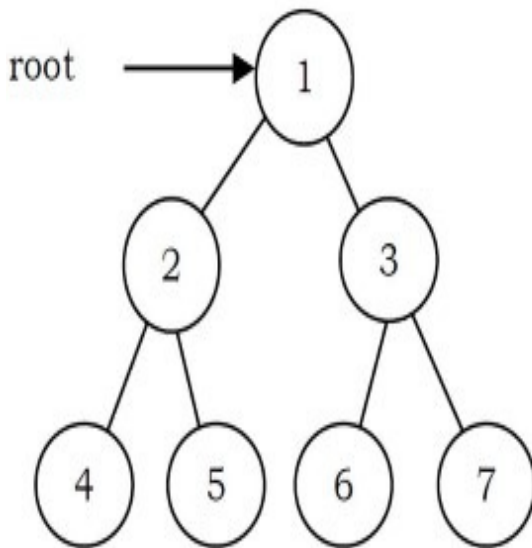
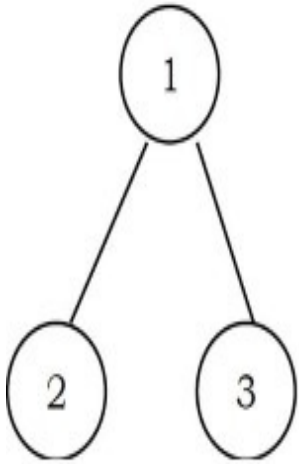
$h = 0$

$2^0 = 1$



$h = 1$

$2^1 = 2$



$h = 2$

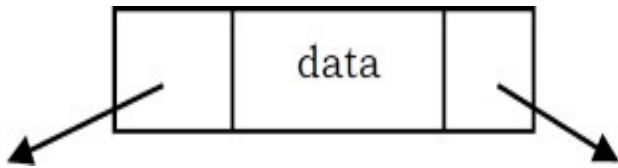
$2^2 = 4$

From the diagram we can infer the following properties:

- The number of nodes n in a full binary tree is $2^{h+1} - 1$. Since, there are h levels we need to add all nodes at each level [$2^0 + 2^1 + 2^2 + \dots + 2^h = 2^{h+1} - 1$].
- The number of nodes n in a complete binary tree is between 2^h (minimum) and $2^{h+1} - 1$ (maximum). For more information on this, refer to [Priority Queues](#) chapter.
- The number of leaf nodes in a full binary tree is 2^h .
- The number of NULL links (wasted pointers) in a complete binary tree of n nodes is $n + 1$.

Structure of Binary Trees

Now let us define structure of the binary tree. For simplicity, assume that the data of the nodes are integers. One way to represent a node (which contains data) is to have two links which point to left and right children along with data fields as shown below:



Or



```
struct BinaryTreeNode {  
    int data;  
    struct BinaryTreeNode *left;  
    struct BinaryTreeNode *right;  
};
```

Note: In trees, the default flow is from parent to children and it is not mandatory to show directed branches. For our discussion, we assume both the representations shown below are the same.



Operations on Binary Trees

Basic Operations

- Inserting an element into a tree
- Deleting an element from a tree
- Searching for an element
- Traversing the tree

Auxiliary Operations

- Finding the size of the tree
- Finding the height of the tree
- Finding the level which has maximum sum
- Finding the least common ancestor (LCA) for a given pair of nodes, and many more.

Applications of Binary Trees

Following are the some of the applications where *binary trees* play an important role:

- Expression trees are used in compilers.
- Huffman coding trees that are used in data compression algorithms.
- Binary Search Tree (BST), which supports search, insertion and deletion on a collection of items in $O(\log n)$ (average).
- Priority Queue (PQ), which supports search and deletion of minimum (or maximum) on a collection of items in logarithmic time (in worst case).

6.6 Binary Tree Traversals

In order to process trees, we need a mechanism for traversing them, and that forms the subject of this section. The process of visiting all nodes of a tree is called *tree traversal*. Each node is processed only once but it may be visited more than once. As we have already seen in linear data structures (like linked lists, stacks, queues, etc.), the elements are visited in sequential order. But, in tree structures there are many different ways.

Tree traversal is like searching the tree, except that in traversal the goal is to move through the tree in a particular order. In addition, all nodes are processed in the *traversal but searching* stops when the required node is found.

Traversal Possibilities

Starting at the root of a binary tree, there are three main steps that can be performed and the order in which they are performed defines the traversal type. These steps are: performing an action on the current node (referred to as “visiting” the node and denoted with “*D*”), traversing to the left child node (denoted with “*L*”), and traversing to the right child node (denoted with “*R*”). This process can be easily described through recursion. Based on the above definition there are 6 possibilities:

1. *LDR*: Process left subtree, process the current node data and then process right subtree
2. *LRD*: Process left subtree, process right subtree and then process the current node data
3. *DLR*: Process the current node data, process left subtree and then process right subtree
4. *DRL*: Process the current node data, process right subtree and then process left subtree
5. *RDL*: Process right subtree, process the current node data and then process left subtree
6. *RLD*: Process right subtree, process left subtree and then process the current node data

Classifying the Traversals

The sequence in which these entities (*nodes*) are processed defines a particular traversal method. The classification is based on the order in which current node is processed. That means, if we are classifying based on current node (*D*) and if *D* comes in the middle then it does not matter whether *L* is on left side of *D* or *R* is on left side of *D*.

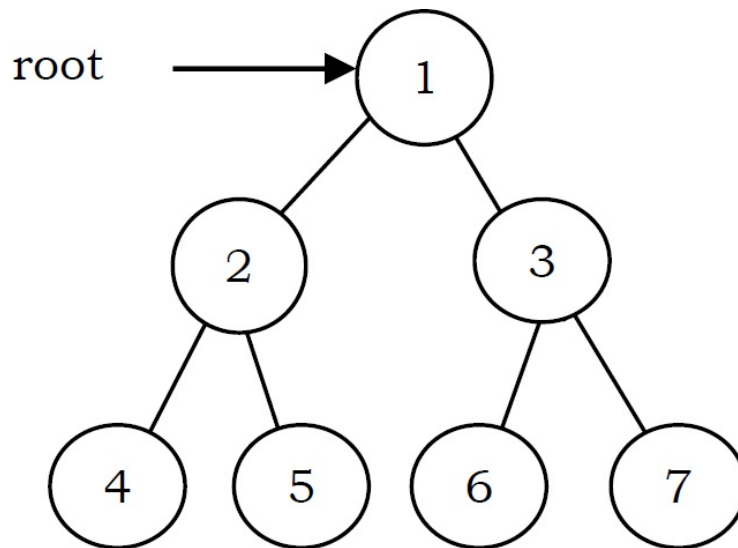
Similarly, it does not matter whether *L* is on right side of *D* or *R* is on right side of *D*. Due to this, the total 6 possibilities are reduced to 3 and these are:

- Preorder (*DLR*) Traversal
- Inorder (*LDR*) Traversal
- Postorder (*LRD*) Traversal

There is another traversal method which does not depend on the above orders and it is:

- Level Order Traversal: This method is inspired from Breadth First Traversal (BFS of Graph algorithms).

Let us use the diagram below for the remaining discussion.



PreOrder Traversal

In preorder traversal, each node is processed before (pre) either of its subtrees. This is the simplest traversal to understand. However, even though each node is processed before the subtrees, it still requires that some information must be maintained while moving down the tree. In the example above, 1 is processed first, then the left subtree, and this is followed by the right subtree.

Therefore, processing must return to the right subtree after finishing the processing of the left subtree. To move to the right subtree after processing the left subtree, we must maintain the root

information. The obvious ADT for such information is a stack. Because of its LIFO structure, it is possible to get the information about the right subtrees back in the reverse order.

Preorder traversal is defined as follows:

- Visit the root.
- Traverse the left subtree in Preorder.
- Traverse the right subtree in Preorder.

The nodes of tree would be visited in the order: 1 2 4 5 3 6 7

```
void PreOrder(struct BinaryTreeNode *root){
    if(root) {
        printf("%d",root->data);
        PreOrder(root->left);
        PreOrder (root->right);
    }
}
```

Time Complexity: $O(n)$. Space Complexity: $O(n)$.

Non-Recursive Preorder Traversal

In the recursive version, a stack is required as we need to remember the current node so that after completing the left subtree we can go to the right subtree. To simulate the same, first we process the current node and before going to the left subtree, we store the current node on stack. After completing the left subtree processing, *pop* the element and go to its right subtree. Continue this process until stack is nonempty.

```

void PreOrderNonRecursive(struct BinaryTreeNode *root){
    struct Stack *S = CreateStack();
    while(1) {
        while(root) {
            //Process current node
            printf("%d",root->data);

            Push(S,root);

            //If left subtree exists, add to stack
            root = root->left;
        }
        if(IsEmptyStack(S))
            break;
        root = Pop(S);
        //Indicates completion of left subtree and current node, now go to right subtree
        root = root->right;
    }
    DeleteStack(S);
}

```

Time Complexity: $O(n)$. Space Complexity: $O(n)$.

InOrder Traversal

In Inorder Traversal the root is visited between the subtrees. Inorder traversal is defined as follows:

- Traverse the left subtree in Inorder.
- Visit the root.
- Traverse the right subtree in Inorder.

The nodes of tree would be visited in the order: 4 2 5 1 6 3 7

```

void InOrder(struct BinaryTreeNode *root){
    if(root) {
        InOrder(root→left);
        printf("%d",root→data);
        InOrder(root→right);
    }
}

```

Time Complexity: $O(n)$. Space Complexity: $O(n)$.

Non-Recursive Inorder Traversal

The Non-recursive version of Inorder traversal is similar to Preorder. The only change is, instead of processing the node before going to left subtree, process it after popping (which is indicated after completion of left subtree processing).

```

void InOrderNonRecursive(struct BinaryTreeNode *root){
    struct Stack *S = CreateStack();
    while(1) {
        while(root) {
            Push(S,root);
            //Got left subtree and keep on adding to stack
            root = root→left;
        }
        if(IsEmptyStack(S))
            break;
        root = Pop(S);
        printf("%d", root→data); //After popping, process the current node
        //Indicates completion of left subtree and current node, now go to right subtree
        root = root→right;
    }
    DeleteStack(S);
}

```

Time Complexity: $O(n)$. Space Complexity: $O(n)$.

PostOrder Traversal

In postorder traversal, the root is visited after both subtrees. Postorder traversal is defined as follows:

- Traverse the left subtree in Postorder.
- Traverse the right subtree in Postorder.
- Visit the root.

The nodes of the tree would be visited in the order: 4 5 2 6 7 3 1

```
void PostOrder(struct BinaryTreeNode *root){
    if(root) {
        PostOrder(root→left);
        PostOrder(root→right);
        printf("%d",root→data);
    }
}
```

Time Complexity: $O(n)$. Space Complexity: $O(n)$.

Non-Recursive Postorder Traversal

In preorder and inorder traversals, after popping the stack element we do not need to visit the same vertex again. But in postorder traversal, each node is visited twice. That means, after processing the left subtree we will visit the current node and after processing the right subtree we will visit the same current node. But we should be processing the node during the second visit. Here the problem is how to differentiate whether we are returning from the left subtree or the right subtree.

We use a *previous* variable to keep track of the earlier traversed node. Let's assume *current* is the current node that is on top of the stack. When *previous* is *current*'s parent, we are traversing down the tree. In this case, we try to traverse to *current*'s left child if available (i.e., push left child to the stack). If it is not available, we look at *current*'s right child. If both left and right child do not exist (ie, *current* is a leaf node), we print *current*'s value and pop it off the stack.

If *prev* is *current*'s left child, we are traversing up the tree from the left. We look at *current*'s right child. If it is available, then traverse down the right child (i.e., push right child to the stack); otherwise print *current*'s value and pop it off the stack. If *previous* is *current*'s right child, we are traversing up the tree from the right. In this case, we print *current*'s value and pop it off the stack.

```

void PostOrderNonRecursive(struct BinaryTreeNode *root) {
    struct SimpleArrayStack *S = CreateStack();
    struct BinaryTreeNode *previous = NULL;
    do{
        while (root!=NULL){
            Push(S, root);
            root = root->left;
        }
        while(root == NULL && !IsEmptyStack(S)){
            root = Top(S);
            if(root->right == NULL || root->right == previous){
                printf("%d ", root->data);
                Pop(S);
                previous = root;
                root = NULL;
            }
            else
                root = root->right;
        }
    }while(!IsEmptyStack(S));
}

```

Time Complexity: $O(n)$. Space Complexity: $O(n)$.

Level Order Traversal

Level order traversal is defined as follows:

- Visit the root.
- While traversing level i , keep all the elements at level $i + 1$ in queue.
- Go to the next level and visit all the nodes at that level.
- Repeat this until all levels are completed.

The nodes of the tree are visited in the order: 1 2 3 4 5 6 7

```

void LevelOrder(struct BinaryTreeNode *root){
    struct BinaryTreeNode *temp;
    struct Queue *Q = CreateQueue();
    if(!root)
        return;
    EnQueue(Q,root);
    while(!IsEmptyQueue(Q)) {
        temp = DeQueue(Q);
        //Process current node
        printf("%d", temp->data);
        if(temp->left)
            EnQueue(Q, temp->left);
        if(temp->right)
            EnQueue(Q, temp->right);
    }
    DeleteQueue(Q);
}

```

Time Complexity: $O(n)$. Space Complexity: $O(n)$. Since, in the worst case, all the nodes on the entire last level could be in the queue simultaneously.

Binary Trees: Problems & Solutions

Problem-1 Give an algorithm for finding maximum element in binary tree.

Solution: One simple way of solving this problem is: find the maximum element in left subtree, find the maximum element in right sub tree, compare them with root data and select the one which is giving the maximum value. This approach can be easily implemented with recursion.

```

int FindMax(struct BinaryTreeNode *root) {
    int root_val, left, right, max = INT_MIN;
    if(root != NULL) {
        root_val = root->data;
        left = FindMax(root->left);
        right = FindMax(root->right);
        // Find the largest of the three values.
        if(left > right)
            max = left;
        else max = right;
        if(root_val > max)
            max = root_val;
    }
    return max;
}

```

Time Complexity: $O(n)$. Space Complexity: $O(n)$.

Problem-2 Give an algorithm for finding the maximum element in binary tree without recursion.

Solution: Using level order traversal: just observe the element's data while deleting.


```

int FindMaxUsingLevelOrder(struct BinaryTreeNode *root){
    struct BinaryTreeNode *temp;
    int max = INT_MIN;
    struct Queue *Q = CreateQueue();
    EnQueue(Q,root);
    while(!IsEmptyQueue(Q)) {
        temp = DeQueue(Q);
        // largest of the three values
        if(max < temp->data)
            max = temp->data;
        if(temp->left)
            EnQueue (Q, temp->left);
        if(temp->right)
            EnQueue (Q, temp->right);
    }
    DeleteQueue(Q);
    return max;
}

```

Time Complexity: $O(n)$. Space Complexity: $O(n)$.

Problem-3 Give an algorithm for searching an element in binary tree.

Solution: Given a binary tree, return true if a node with data is found in the tree. Recurse down the tree, choose the left or right branch by comparing data with each node's data.

```

int FindInBinaryTreeUsingRecursion(struct BinaryTreeNode *root, int data) {
    int temp;
    // Base case == empty tree, in that case, the data is not found so return false
    if(root == NULL)
        return 0;
    else {
        // see if found here
        if(data == root->data)
            return 1;
        else {
            // otherwise recur down the correct subtree
            temp = FindInBinaryTreeUsingRecursion (root->left, data)
            if(temp != 0)
                return temp;
            else return(FindInBinaryTreeUsingRecursion(root->right, data));
        }
    }
    return 0;
}

```

Time Complexity: $O(n)$. Space Complexity: $O(n)$.

Problem-4 Give an algorithm for searching an element in binary tree without recursion.

Solution: We can use level order traversal for solving this problem. The only change required in level order traversal is, instead of printing the data, we just need to check whether the root data is equal to the element we want to search.

```

int SearchUsingLevelOrder(struct BinaryTreeNode *root, int data){
    struct BinaryTreeNode *temp;
    struct Queue *Q;
    if(!root) return -1;
    Q = CreateQueue();
    EnQueue(Q,root);
    while(!IsEmptyQueue(Q)) {
        temp = DeQueue(Q);
        // see if found here
        if(data == root->data)
            return 1;
        if(temp->left)
            EnQueue (Q, temp->left);
        if(temp->right)
            EnQueue (Q, temp->right);
    }
    DeleteQueue(Q);
    return 0;
}

```

Time Complexity: $O(n)$. Space Complexity: $O(n)$.

Problem-5 Give an algorithm for inserting an element into binary tree.

Solution: Since the given tree is a binary tree, we can insert the element wherever we want. To insert an element, we can use the level order traversal and insert the element wherever we find the node whose left or right child is NULL.

```

void InsertInBinaryTree(struct BinaryTreeNode *root, int data){
    struct Queue *Q;
    struct BinaryTreeNode *temp;
    struct BinaryTreeNode *newNode;
    newNode = (struct BinaryTreeNode *) malloc(sizeof(struct BinaryTreeNode));
    newNode->left = newNode->right = NULL;
    if(!newNode) {
        printf("Memory Error"); return;
    }
    if(!root) {
        root = newNode;
        return;
    }
    Q = CreateQueue();
    EnQueue(Q,root);

    while(!IsEmptyQueue(Q)) {
        temp = DeQueue(Q);
        if(temp->left)
            EnQueue(Q, temp->left);
        else {
            temp->left=newNode;
            DeleteQueue(Q);
            return;
        }
        if(temp->right)
            EnQueue(Q, temp->right);
        else {
            temp->right=newNode;
            DeleteQueue(Q);
            return;
        }
    }
    DeleteQueue(Q);
}

```

Time Complexity: $O(n)$. Space Complexity: $O(n)$.

Problem-6 Give an algorithm for finding the size of binary tree.

Solution: Calculate the size of left and right subtrees recursively, add 1 (current node) and return to its parent.

```
// Compute the number of nodes in a tree.
int SizeOfBinaryTree(struct BinaryTreeNode *root) {
    if(root==NULL)
        return 0;
    else return(SizeOfBinaryTree(root→left) + 1 + SizeOfBinaryTree(root→right));
}
```

Time Complexity: $O(n)$. Space Complexity: $O(n)$.

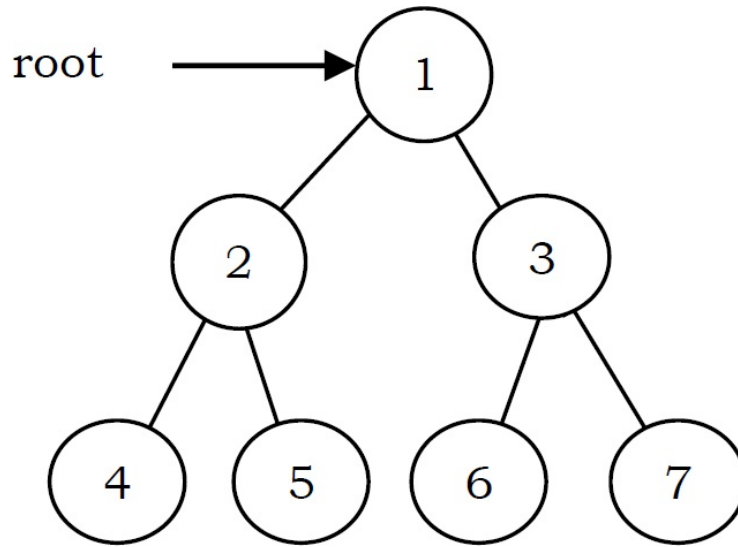
Problem-7 Can we solve [Problem-6](#) without recursion?

Solution: Yes, using level order traversal.

```
int SizeofBTUsingLevelOrder(struct BinaryTreeNode *root){
    struct BinaryTreeNode *temp;
    struct Queue *Q;
    int count = 0;
    if(!root) return 0;
    Q = CreateQueue();
    EnQueue(Q,root);
    while(!IsEmptyQueue(Q)) {
        temp = DeQueue(Q);
        count++;
        if(temp→left)
            EnQueue (Q, temp→left);
        if(temp→right)
            EnQueue (Q, temp→right);
    }
    DeleteQueue(Q);
    return count;
}
```

Time Complexity: $O(n)$. Space Complexity: $O(n)$.

Problem-8 Give an algorithm for printing the level order data in reverse order. For example, the output for the below tree should be: 4 5 6 7 2 3 1



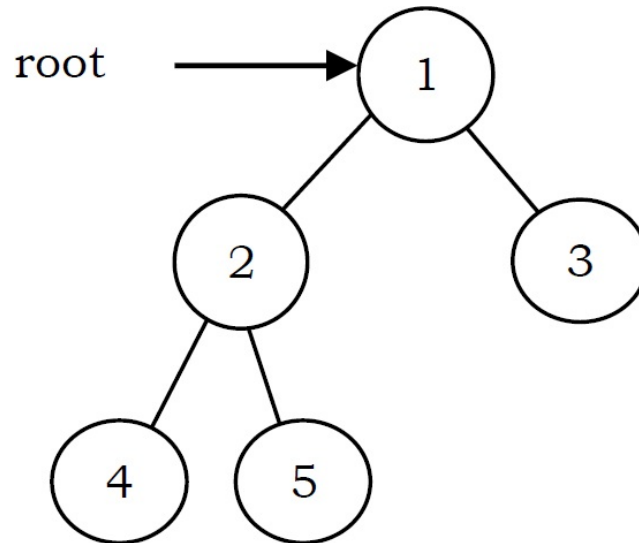
Solution:

```
void LevelOrderTraversalInReverse(struct BinaryTreeNode *root){
    struct Queue *Q;
    struct Stack *s = CreateStack();
    struct BinaryTreeNode *temp;
    if(!root) return;
    Q = CreateQueue();
    EnQueue(Q, root);
    while(!IsEmptyQueue(Q)) {
        temp = DeQueue(Q);
        if(temp->right)
            EnQueue(Q, temp->right);
        if(temp->left)
            EnQueue(Q, temp->left);
        Push(s, temp);
    }
    while(!IsEmptyStack(s))
        printf("%d", Pop(s)->data);
}
```

Time Complexity: $O(n)$. Space Complexity: $O(n)$.

Problem-9 Give an algorithm for deleting the tree.

Solution:



To delete a tree, we must traverse all the nodes of the tree and delete them one by one. So which traversal should we use: Inorder, Preorder, Postorder or Level order Traversal?

Before deleting the parent node we should delete its children nodes first. We can use postorder traversal as it does the work without storing anything. We can delete tree with other traversals also with extra space complexity. For the following, tree nodes are deleted in order – 4,5,2,3,1.

```
void DeleteBinaryTree(struct BinaryTreeNode *root){
    if(root == NULL)
        return;
    /* first delete both subtrees */
    DeleteBinaryTree(root->left);
    DeleteBinaryTree(root->right);
    //Delete current node only after deleting subtrees
    free(root);
}
```

Time Complexity: $O(n)$. Space Complexity: $O(n)$.

Problem-10 Give an algorithm for finding the height (or depth) of the binary tree.

Solution: Recursively calculate height of left and right subtrees of a node and assign height to the node as max of the heights of two children plus 1. This is similar to *PreOrder* tree traversal (and *DFS* of Graph algorithms).


```

int HeightOfBinaryTree(struct BinaryTreeNode *root){
    int leftheight, rightheight;
    if(root == NULL)
        return 0;
    else {
        /* compute the depth of each subtree */
        leftheight = HeightOfBinaryTree(root→left);
        rightheight = HeightOfBinaryTree(root→right);

        if(leftheight > rightheight)
            return(leftheight + 1);
        else
            return(rightheight + 1);
    }
}

```

Time Complexity: $O(n)$. Space Complexity: $O(n)$.

Problem-11 Can we solve [Problem-10](#) without recursion?

Solution: Yes, using level order traversal. This is similar to *BFS* of Graph algorithms. End of level is identified with NULL.


```

int FindHeightofBinaryTree(struct BinaryTreeNode *root){
    int level = 0;
    struct Queue *Q;
    if(!root) return 0;
    Q = CreateQueue();
    EnQueue(Q,root);
    // End of first level
    EnQueue(Q,NULL);
    while(!IsEmptyQueue(Q)) {
        root=DeQueue(Q);
        // Completion of current level.
        if(root==NULL) {
            //Put another marker for next level.
            if(!IsEmptyQueue(Q))
                EnQueue(Q,NULL);
            level++;
        }
        else {
            if(root→left)
                EnQueue(Q, root→left);
            if(root→right)
                EnQueue(Q, root→right);
        }
    }
    return level;
}

```

Time Complexity: $O(n)$. Space Complexity: $O(n)$.

Problem-12 Give an algorithm for finding the deepest node of the binary tree.

Solution:

```

struct BinaryTreeNode *DeepestNodeinBinaryTree(struct BinaryTreeNode *root){
    struct BinaryTreeNode *temp;
    struct Queue *Q;
    if(!root) return NULL;
    Q = CreateQueue();
    EnQueue(Q,root);
    while(!IsEmptyQueue(Q)) {
        temp = DeQueue(Q);
        if(temp→left)
            EnQueue(Q, temp→left);
        if(temp→right)
            EnQueue(Q, temp→right);
    }
    DeleteQueue(Q);
    return temp;
}

```

Time Complexity: $O(n)$. Space Complexity: $O(n)$.

Problem-13 Give an algorithm for deleting an element (assuming data is given) from binary tree.

Solution: The deletion of a node in binary tree can be implemented as

- Starting at root, find the node which we want to delete.
- Find the deepest node in the tree.
- Replace the deepest node's data with node to be deleted.
- Then delete the deepest node.

Problem-14 Give an algorithm for finding the number of leaves in the binary tree without using recursion.

Solution: The set of nodes whose both left and right children are NULL are called leaf nodes.